

INTERLEAVE

**Two concurrency defects,
traced and checked after repair.**

A Coyote-backed .NET 8 proof of concept

Both baselines violated a business invariant

Coyote found schedules where concurrent requests duplicated a scarce resource.

LAST SEAT

2

**successful reservations
for one available seat**

Baseline report: 5 paths explored, 1 invariant violation.

Invariant: reservations cannot exceed capacity.

DUPLICATE JOB CLAIM

2

**workers returned
Claimed for one job**

Baseline report: 8 fair paths explored, 1 invariant violation.

Invariant: at most one worker may claim the job.

A frozen failure makes the repair testable

Coyote supplies systematic schedules and the invariant verdict for each example.

01

Define

Write the business invariant as a test.

02

Explore

Coyote controls concurrent schedules.

03

Preserve

Keep the failing report, raw trace, and checksum.

04

Recheck

Repair the state change, then run bounded checks.

IBM Bob assisted development. Microsoft Coyote controls exploration and reports test results.

Both repairs passed bounded local exploration

Coyote 1.7.11 used the portfolio fair strategy and a 1,000-step scheduling limit.

LAST SEAT

500

fair paths

0

bugs found

Repaired bounded run

DUPLICATE JOB CLAIM

500

fair paths

0

bugs found

Repaired bounded run

Local suites passed: 13 / 13 sequential tests and 9 / 9 Coyote tests.

Finite schedule coverage only. These runs do not prove every execution is safe.

Each lock protects the complete state change

The invariant checks the actual business outcome, not a counter that can hide a race.

LAST SEAT

One lock spans the reservation path.

Availability check
Seat decrement
Reservation record

Reads synchronize; callers receive detached reservation snapshots.

DUPLICATE JOB CLAIM

One lock spans check and claim.

Check job status
Mark the job claimed
Return the worker outcome

Test counts actual `ClaimResult.Claimed` returns.

A tested local prototype still has release gates

Current evidence describes two in-memory .NET examples, not a hosted product.

VERIFIED LOCALLY

Two synthetic .NET 8 concurrency scenarios
Frozen Coyote baseline reports, traces, and checksums
13 / 13 sequential tests and 9 / 9 Coyote tests
500 fair paths per repaired scenario at a 1,000-step limit

STILL OPEN

Verified release tag
Genuine Bob session-summary screenshots
Hosted user-facing demo and application URL
Recorded MP4 with a real working-product demo
Remote CI, benchmark, and event eligibility checks

Exact schedule alignment for both supplied post-fix traces remains unresolved. Bounded results do not guarantee race freedom.